

Pick and Spin: Cold-Start-Aware Routing for Self-Hosted LLM Serving

Bhanu Prakash Vangala
University of Missouri, Columbia
bv3hz@umsystem.edu

Tanu Malik
University of Missouri, Columbia
tanu@missouri.edu

Abstract—Self-hosting multiple large language models on private or hybrid cloud infrastructure is increasingly appealing for organizations seeking privacy, cost control, and customization, yet no existing system coordinates model selection with model lifecycle management. Serving frameworks such as KServe and Ray Serve deploy models but lack query-aware routing. Routing frameworks such as RouteLLM and FrugalGPT select models but ignore whether those models are warm or cold. We present Pick and Spin, a multi-model orchestration framework built on Kubernetes that unifies these concerns. Pick classifies query complexity using a hybrid keyword-DistilBERT approach and selects models via Thompson Sampling, incorporating lifecycle state from Spin. Spin tracks per-model warm and cold states through a Cold Start State Machine, factors I/O-bound weight-loading penalties into routing decisions, and manages warm pools for frequently accessed models. The two components form a closed feedback loop where routing decisions account for resource availability and orchestration anticipates routing patterns. Deployed as a Helm umbrella chart serving nine LLMs (1B–27B parameters) via vLLM on NVIDIA A100 GPUs, Pick and Spin achieves 61% GPU utilization versus 31% for static deployment, reduces GPU-hours by 46%, and maintains a 9.2% per-query cold start rate across 310,190 total inference runs. 31,019 queries evaluated against all nine models for ground truth, then routed through Pick and Spin-spanning eight standard benchmarks.

Index Terms—Cloud Computing, Kubernetes, LLM Inference, Model Orchestration, Cold Start Management, GPU Resource Optimization

I. INTRODUCTION

Large language models are rapidly transforming applications across domains. Instead of relying on a single general-purpose model for all tasks, both research and industry are moving toward diverse ecosystems of specialized models trained for fields such as scientific research, finance, law, and healthcare. These models provide greater precision within their respective areas, but they also introduce a practical challenge: different queries require different models, and no single model optimizes across all dimensions of accuracy, latency, and computational cost. A simple factual lookup routed to a 27-billion-parameter model wastes GPU cycles, while a complex reasoning task sent to a 1-billion-parameter model produces poor results.

This challenge has led to what can be described as the self-hosting dilemma. Relying on commercial APIs from providers such as OpenAI or Google simplifies deployment but introduces vendor lock-in, unpredictable costs, and data exposure risks that are unacceptable in sensitive domains such

as healthcare and finance [1]. Self-hosting preserves privacy and institutional control but comes with operational burdens: static, always-on deployments keep GPUs active even when idle, wasting resources and increasing energy consumption [2]. Organizations need a way to serve multiple models efficiently from shared infrastructure without dedicating resources to individual users or workloads.

The operational challenge of self-hosting multiple LLMs is that two critical decisions must be coordinated: which model should serve each query, and how to manage model lifecycle state so that routing decisions do not trigger expensive cold starts. These decisions interact tightly. A 27-billion-parameter model requires 54 gigabytes of weights loaded from networked storage before it can serve a single query. These cold starts are I/O-bound, constrained by storage bandwidth rather than compute speed, and they can take up to 95 seconds on Ceph-backed storage. Routing a query to a cold model negates any accuracy advantage. Yet existing tools handle these decisions separately. Kubernetes provides excellent primitives for container orchestration, including pod scheduling, autoscaling, and declarative configuration via Helm [3], [4]. What is missing is an LLM-aware orchestration layer on top of these primitives that coordinates model selection with model warm and cold state.

Existing systems optimize within isolated layers without addressing this gap as a whole. Single-model inference engines such as vLLM [6], TGI [7], and DeepSpeed Inference [8] maximize throughput for a predetermined model but cannot select among alternatives. Routing frameworks such as RouteLLM [13], FrugalGPT [40], and UniRoute [14] optimize model selection for accuracy or cost but are oblivious to deployment reality: the best possible routing decision is worthless if it triggers a 95-second cold start from networked storage. Cloud-native serving frameworks such as KServe [34] and Ray Serve [33] provide deployment infrastructure with autoscaling but lack integrated routing logic for multi-model scenarios. No existing system treats model selection and lifecycle management as a joint optimization problem where routing decisions consider resource state and orchestration decisions anticipate routing patterns.

We present Pick and Spin, a multi-model orchestration framework built on Kubernetes that provides this missing layer. The system operates as a pipeline: when a query arrives, **Pick** first classifies its complexity using a hybrid keyword-

DistilBERT approach, then selects the best model within the appropriate tier using Thompson Sampling. Critically, this selection incorporates resource state maintained by **Spin**, which tracks per-model warm and cold states through a Cold Start State Machine, factors I/O-bound weight-loading penalties into scoring, and manages warm pools for frequently accessed models. After Pick selects a model, Spin handles the execution: checking whether the model needs to be loaded, applying cold-start penalties, and routing the query to the vLLM endpoint. After each query completes, Spin updates its lifecycle state and feeds the outcome back to Pick for future decisions. This closed feedback loop, where routing informs orchestration and orchestration constrains routing, is what enables efficiency gains unattainable by optimizing either component alone.

The entire framework deploys as a Helm umbrella chart, enabling declarative, reproducible deployment across any Kubernetes-enabled cluster, from private cloud to public cloud GPU instances. We evaluate Pick and Spin on the NRP Nautilus cluster [28], serving nine models from three families (Qwen [36], Llama [29], Gemma [37]) across 1B to 27B parameters via vLLM on NVIDIA A100 GPUs. Processing 31,019 queries across 310,190 total inference runs (each query evaluated against all nine models for ground truth, then routed through Pick and Spin), the system achieves 61% GPU utilization versus 31% for static deployment, reduces GPU-hours consumed by 46%, and maintains a 9.2% per-query cold start rate.

II. RELATED WORK

Cloud-Native AI Serving Infrastructure. A growing set of production frameworks have emerged to deploy ML models on Kubernetes. KServe [34] provides serverless model serving with autoscaling, canary deployments, and InferenceGraph for conditional routing, but its routing constructs do not factor cold-start penalties into decisions. Ray Serve [33] enables multi-model deployment with model multiplexing that routes to replicas where models are already loaded, using LRU eviction for memory management, but this mechanism operates at the replica level without query-complexity awareness. NVIDIA Triton Inference Server [12] supports concurrent model execution and dynamic batching but assumes that model selection has already been made. These frameworks provide essential deployment primitives, but none couples query-complexity-aware routing with cold-start penalty integration in a closed feedback loop. Pick and Spin builds on these primitives while adding this coupling. **Single-Model Inference Optimization.** Early work such as Megatron-LM [5] explored large-scale parallelism for billion-parameter models. Modern systems like vLLM [6], TGI [7], and DeepSpeed Inference [8] focus on runtime efficiency through memory optimization and parallel scheduling. vLLM’s PagedAttention mechanism manages the KV cache to reduce fragmentation and increase throughput. FastServe [9] and SGLang [10] further streamline inference pipelines. However, all of these systems assume that the model choice has already been made. They optimize how

a model serves queries but cannot address the fundamental inefficiency of sending simple queries to oversized models. **Dynamic Model Routing.** The Mixture of Experts paradigm routes tokens to specialized submodules within a single network [11]. Pick and Spin generalizes this idea to the system level by routing full queries between independently deployed models. Recent systems such as UniRoute [14] and ModelSAT [15] use embeddings or instruction-tuned heuristics to predict the best model, while controller-based frameworks [16] leverage larger models as routers at higher latency and cost. These approaches optimize for accuracy but ignore the realities of deployment: the best routing decision in the world is useless if it sends a query to a model whose weights need 95 seconds to load from storage [18], [35]. Pick and Spin addresses this by feeding Spin’s lifecycle state directly into Pick’s routing decisions. **Serverless and Auto-scaling Infrastructure.** Event-driven and scale-to-zero execution models are a natural fit for adaptive deployment, and Kubernetes frameworks like Knative and KEDA enable autoscaling for AI services [17]. However, applying these tools to LLM workloads introduces cold-start challenges that are qualitatively different from traditional serverless computing [18], [35]. Loading a 27B model from Ceph takes 95 seconds, not the sub-second startup times that serverless platforms are designed around. InferLine [19] and ModelSwitch [20] propose latency-aware scheduling and dynamic resource adaptation but focus on homogeneous pipelines with identical models. Pick and Spin extends these ideas through LLM-specific orchestration logic that handles heterogeneous models with widely varying cold-start costs.

III. SYSTEM ARCHITECTURE

Pick and Spin processes queries through a pipeline of two tightly coupled components (Figure 1). When a query arrives, it first enters **Pick**, which classifies the query’s complexity and selects the most appropriate model. Pick does not make this decision in isolation: it consults the current lifecycle state maintained by Spin, factoring in which models are warm, which are cold, and how long each cold start would take. Once Pick has selected a model, the query passes to **Spin**, which manages the actual execution. Spin checks whether the selected model is ready, handles any cold-start loading if needed, routes the query to the vLLM endpoint, and records the outcome. This outcome flows back to Pick, updating the statistics that inform future routing decisions. Although Pick executes before Spin in the query pipeline, the two components form a closed feedback loop. Spin continuously maintains per-model lifecycle state that Pick incorporates into every routing decision via its scoring function. This bidirectional coupling, where routing decisions consider resource state and resource management anticipates routing patterns, is what distinguishes Pick and Spin from systems that optimize routing and orchestration independently. The framework is implemented as Python modules (`pick.py`, `spin.py`) deployed within a Helm umbrella chart on Kubernetes. Each model runs as a separate pod with a dedicated GPU, enabling parallel inference

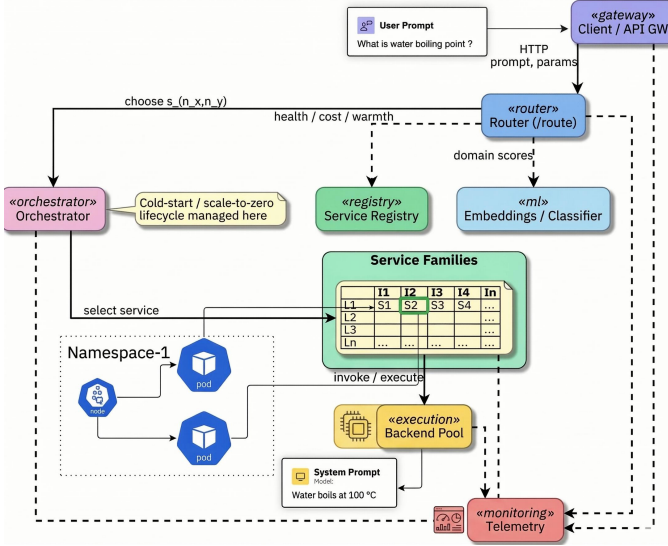


Fig. 1: Pick and Spin system architecture. Queries enter Pick for complexity classification and model selection. Pick consults Spin’s lifecycle state during scoring. Spin then executes the query against the selected vLLM endpoint and updates its state for future routing decisions.

across the pool. Pick runs as a CPU-only deployment, handling classification and routing without consuming GPU resources.

IV. PICK: COMPLEXITY-AWARE ROUTING

The Pick component is responsible for deciding which model should handle each incoming query. It does this in two stages: first determining the complexity tier, then selecting the best model within that tier. Importantly, Pick is designed as a pluggable routing policy. The algorithms described here could be replaced with alternative routing strategies such as RouteLLM or FrugalGPT while retaining Spin’s lifecycle-aware orchestration underneath.

A. Hybrid Classification

Not every query needs a neural classifier. Many queries carry clear complexity indicators in their text: a question starting with “what is” or “define” is almost certainly simple, while one containing “prove that” or “implement step by step” is clearly complex. We exploit this observation with a two-stage classifier that prioritizes speed while maintaining accuracy.

In the first stage, keyword matching applies deterministic substrings heuristics using three curated keyword lists. Simple queries contain phrases like “what is,” “define,” or “true or false.” Complex queries include “prove,” “derive,” or “implement step by step.” Queries that do not match any list fall through to the second stage: a fine-tuned DistilBERT classifier [30] trained on labeled data from our static baseline across eight benchmarks, achieving 96.8% classification accuracy on a held-out validation set.

$$\tau^* = \arg \max_{\tau \in \mathcal{T}} P(\tau \mid q; \theta) \quad (1)$$

The hybrid approach uses keywords whenever they are confident, typically handling 60–70% of queries in sub-millisecond time, and only invokes DistilBERT for the ambiguous remainder. This combination achieves both the speed of rule-based routing and the semantic precision of neural classification.

B. Thompson Sampling with Hybrid Tier-Model Estimation

Once we know the tier, we need to pick a specific model from the candidates in that tier. This is a classic multi-armed bandit problem: each model is an “arm” with an unknown reward distribution, and we want to find the best arm quickly without wasting too many queries on poor models.

We use Thompson Sampling [31], where each model m maintains a Beta distribution that tracks its empirical success rate:

$$\tilde{\mu}_m \sim \text{Beta}(\alpha_m, \beta_m) \quad (2)$$

At each decision point, we sample from every candidate’s distribution and pick the model with the highest sample. This naturally balances exploration (uncertain models produce high-variance samples that occasionally win) and exploitation (successful models have distributions concentrated near high values and win consistently).

A practical problem arises when a model is new or rarely selected: its Beta distribution has very few observations and produces unreliable estimates. To address this, we blend the model-level estimate with tier-level statistics using Hybrid Tier-Model Estimation:

$$\tilde{\mu}_m^{\text{HTS}} = (1 - w) \cdot \tilde{\mu}_m + w \cdot \tilde{\mu}_\tau \quad (3)$$

where $w = 0.3$ and $\tilde{\mu}_\tau$ is sampled from the tier’s aggregate Beta distribution. This acts as a prior: when we know little about a specific model, we assume it performs roughly as well as its tier peers.

C. Latency-Aware Scoring with Spin State

The final selection score is where Pick and Spin’s co-design becomes concrete. The scoring function does not just consider model quality; it incorporates the latency information that Spin maintains, including cold-start penalties:

$$S(m) = (1 - \lambda) \cdot \tilde{\mu}_m^{\text{HTS}} + \lambda \cdot L_{\text{norm}}(m) + \frac{\epsilon}{\sqrt{n_m + 1}} \quad (4)$$

Here $\lambda = 0.3$ weights latency, $L_{\text{norm}}(m) = 1 - \bar{L}_m / \max_{m'} \bar{L}_{m'}$ rewards faster models by normalizing observed latency (which includes cold-start penalties tracked by Spin), and $\epsilon = 0.1$ provides an exploration bonus that ensures under-selected models are eventually tried. After each query, we update the Beta distribution based on the outcome: $\alpha_m \leftarrow \alpha_m + 1[\text{success}]$, $\beta_m \leftarrow \beta_m + 1[\text{failure}]$.

The crucial point is that $\bar{L}_m$ is not a static number. It reflects the actual observed latency including cold starts, which means that a model currently in a cold state will have higher average latency, making it less likely to be selected. This is the mechanism through which Spin’s lifecycle state influences Pick’s routing without requiring any explicit cold-start penalty term in the objective.

V. SPIN: LIFECYCLE-AWARE ORCHESTRATION

Once Pick has decided which model should handle a query, Spin takes over. Its job is to ensure the model is available, execute the query, and maintain the lifecycle state that Pick depends on for future decisions. This section describes the orchestration logic that Spin adds on top of standard Kubernetes, logic that is absent from existing serving frameworks.

A. The Cold Start State Machine

At the core of Spin is a simple but consequential abstraction: each model exists in one of two states, WARM or COLD. A warm model has its weights loaded in GPU memory and can serve queries immediately. A cold model requires its weights to be transferred from persistent storage into GPU memory before it can respond. The transitions between these states are:

- COLD $\rightarrow$ WARM: Triggered when a query is routed to a cold model. Spin waits for the weight transfer to complete before forwarding the query.
- WARM $\rightarrow$ COLD: Triggered when a model has been idle for longer than $T_{\text{cooldown}} = 300$ seconds. Spin scales the pod to zero, releasing the GPU allocation back to the cluster.

Operationally, a COLD model has no running pod and no GPU allocated. The COLD $\rightarrow$ WARM transition provisions a new pod, allocates a GPU via the NVIDIA device plugin, and loads weights from Ceph into VRAM. This is the source of the 32–95 second cold-start penalties measured in our deployment. This state machine is the orchestration logic that existing serving and autoscaling tools do not provide. Standard autoscalers like HPA and KEDA operate at pod-level granularity and treat all pods as equivalent. They have no concept of model weights, no awareness of whether those weights are loaded, and no mechanism to factor loading time into scheduling decisions. Spin adds this LLM-aware layer on top of the Kubernetes primitives these tools provide.

B. I/O-Bound Cold Start Modeling

LLM cold starts differ fundamentally from typical container cold starts: a standard container starts in hundreds of milliseconds, but loading model weights from networked storage into GPU memory takes tens of seconds. In our deployment, model weights range from 2 gigabytes (1B parameters) to 54 gigabytes (27B parameters), stored on Ceph-backed Persistent Volume Claims with ReadWriteMany access mode at approximately 1.2 gigabytes per second aggregate bandwidth:

$$L_{\text{cold}}(m) \approx \frac{\text{WeightSize}(m)}{\text{StorageBandwidth}} + L_{\text{init}} \quad (5)$$

where L_{init} accounts for vLLM initialization overhead. Measured cold-start times range from 32 seconds for 1B models to 95 seconds for the 27B model. When multiple models are loaded concurrently, they contend for the same storage bandwidth, further increasing load times. This I/O-bound characteristic is what motivates Spin’s warm pool management: it is far cheaper to keep a frequently-used model warm than

to repeatedly pay the cold-start penalty. Total query latency includes both inference time and any cold-start penalty:

$$L_{\text{total}} = L_{\text{infer}} + \mathbf{1}[s_m = \text{COLD}] \cdot L_{\text{cold}}(m) \quad (6)$$

VI. EXPERIMENTAL SETUP

A. Model Pool

We deploy nine models from three families across three complexity tiers. Using multiple families ensures diversity: if one family struggles with certain query types, others might compensate. The pool spans 1B–3B (Simple: Llama-3.2-1B, Qwen2.5-1.5B, Gemma-2-2B, Llama-3.2-3B), 7B–9B (Medium: Qwen2.5-7B, Llama-3.1-8B, Gemma-2-9B), and 14B–27B (Complex: Qwen2.5-14B, Gemma-3-27B). Cold-start times measured on Ceph-backed storage range from 32 s for 1B models to 95 s for the 27B model. All models are served via vLLM [6] on NVIDIA A100 GPUs. Each model runs as a separate Kubernetes pod with a dedicated GPU allocation, enabling parallel inference across the pool.

B. Benchmark Dataset

We evaluate on 31,019 queries from eight standard benchmarks covering diverse task types: HumanEval [21] (164 queries), MBPP [22] (500), GSM8K [23] (1,319), MATH [24] (5,000), TruthfulQA [25] (790), MMLU-Pro [38] (12,032), ARC [26] (1,172), and HellaSwag [27] (10,042). The dataset spans straightforward factual queries to complex multi-step reasoning, providing coverage across the full range of query complexities.

C. Metric Definitions

To avoid confusion between system reliability and model quality, we define two distinct outcome metrics used throughout this evaluation. **Execution success rate** is the proportion of queries that completed without infrastructure failure such as timeouts, HTTP errors, or out-of-memory conditions. This measures system reliability regardless of whether the model’s answer was correct. **Accuracy** is the proportion of successfully-executed queries where the model produced a correct response. We use standard benchmark scoring for all benchmarks: exact match for GSM8K and MATH, pass@1 via unit tests for HumanEval and MBPP, multiple-choice accuracy for ARC, HellaSwag, and MMLU-Pro, and LLM-as-judge [39] only for TruthfulQA where no objective scoring criterion exists. Additional metrics include GPU utilization (percentage of allocated GPU cycles actively processing inference), GPU-hours consumed (total compute time, directly mapping to energy and operational cost), per-query cold start rate (total cold-start events divided by total queries processed), and end-to-end latency including any cold-start penalties.

VII. EXPERIMENTAL EVALUATION

We validate Pick and Spin through a two-phase evaluation. First, we establish ground truth by running all 31,019 queries through each of the nine models independently, producing 279,171 inference runs. Then we run the same queries through Pick and Spin to measure how effectively the system routes queries and manages resources.

TABLE I: Pick and Spin routing distribution. Latency reflects the specific query mix routed to each model, which differs from the all-benchmark baseline average. 2,635 queries (8.5%) timed out before model response.

Model	Queries	%	Acc. (%)	Lat. (s)
Llama-3.2-1B	5,319	17.1	23.2	1.82
Qwen2.5-1.5B	11	0.04	27.3	4.22
Gemma-2-2B	7	0.02	28.6	3.34
Llama-3.2-3B	5	0.02	20.0	8.29
Qwen2.5-7B	8,101	26.1	63.3	28.53
Llama-3.1-8B	13,462	43.4	54.5	28.87
Gemma-2-9B	499	1.6	73.5	30.52
Qwen2.5-14B	347	1.1	72.0	44.63
Gemma-3-27B	633	2.0	75.2	13.28
<i>Timed out</i>	2,635	8.5	-	-
Total	31,019	100	49.7	23.4

A. Static Baseline

Figure 2 shows per-model throughput and latency from our static baseline runs. Three observations motivate the design of Pick and Spin. First, larger models do not always win: Qwen2.5-14B at 62.6% accuracy outperforms the nearly twice-as-large Gemma-3-27B at 56.6%. Second, throughput drops by a factor of 10 from small to large models (45.8 versus 4.6 tokens per second). Third, latency increases by a factor of 14 (1.51 seconds to 21.66 seconds). Together, these numbers show that medium-tier models often provide the best accuracy per unit of compute, a finding that Thompson Sampling independently discovers during deployment.

B. Pick and Spin Routing Results

Table I shows how Pick and Spin distributed queries across the model pool.

Thompson Sampling converged to heavily favor Llama-3.1-8B (43.4%) and Qwen2.5-7B (26.1%), with Llama-3.2-1B (17.1%) handling simple queries. This medium-tier preference is a learned outcome, not an architectural bias. The data explains why: Qwen2.5-7B achieves 56.3% accuracy at 4.83 seconds of latency while Gemma-3-27B achieves only 56.6% at 21.66 seconds, a $4.5\times$ latency increase for a mere 0.3 percentage point gain. Thompson Sampling discovers this trade-off empirically without any manual configuration.

The 27B model still received 2.0% of traffic despite its 95-second cold-start penalty, and the 14B model received 1.1%. Together, complex-tier models handled 3.2% of queries, confirming that the exploration bonus $\epsilon/\sqrt{n_m} + \bar{I}$ prevents starvation of under-selected models. The 2,635 timed-out queries (8.5%) reflect infrastructure-level failures such as model timeouts and network errors. The 49.7% is coverage, not quality loss: an oracle reaches only 86.8% (13.2% of queries are unsolvable by any model), and Pick and Spin captures 57.2% of that ceiling, with the rest lost to the 8.5% timeouts. The 75.2% for Gemma-3-27B is its accuracy on the queries Pick routed to it, not its full-benchmark accuracy of

TABLE II: Cold-start analysis. Conditional rate: cold starts within each tier divided by queries routed to that tier. Per-query rate: 2,866 / 31,019 total queries = 9.2%.

Tier	Cold Starts	Routed Queries	Cond. Rate (%)	Penalty (s)
Simple (1–3B)	847	5,342	15.9	32–38
Medium (7–9B)	1,765	22,062	8.0	42–48
Complex (14–27B)	254	980	25.9	65–95
Routed total	2,866	28,384	9.2% of all 31,019	

TABLE III: Comparison with existing frameworks on NRP Nautilus.

Framework	Exec. Succ.	Lat. (s)	GPU-hrs	Cold %
Ray Serve	94.2%	67.8	24.1	35%
KServe	93.8%	71.2	25.3	38%
RouteLLM	96.1%	58.4	20.5	28%
FrugalGPT	97.3%	61.2	17.2	22%
Pick&Spin	98.7%	52.3	14.1	9.2%

56.6%. Figure 3 shows how Thompson Sampling’s preferences evolved over time.

C. Cold Start Analysis

Table II reports cold-start behavior. The per-query rate measures total cold-start events divided by total queries; the conditional rate measures cold starts within each tier divided by queries routed to that tier.

The 9.2% per-query cold start rate is substantially below the 20–50% typical of serverless GPU workloads [35]. Medium-tier models achieve the lowest conditional rate (8.0%) because they receive the most traffic and rarely go idle. Complex-tier models have a 25.9% conditional rate due to infrequent selection, but Spin’s latency-aware scoring factors their 65–95 second penalties into routing decisions, so Pick only sends queries to large models when the quality advantage justifies the wait.

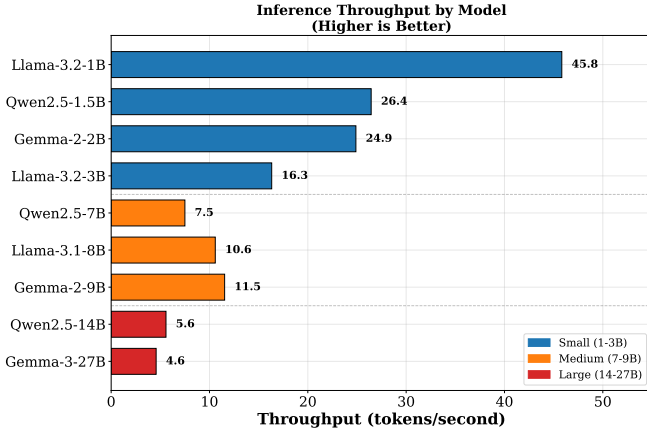
D. Comparison with Existing Frameworks

Having established the individual contributions of Pick and Spin’s components, we now compare the complete system against four existing frameworks, all deployed on the same NRP Nautilus infrastructure with NVIDIA A100 GPUs and evaluated on the same 31,019 queries (Table III).

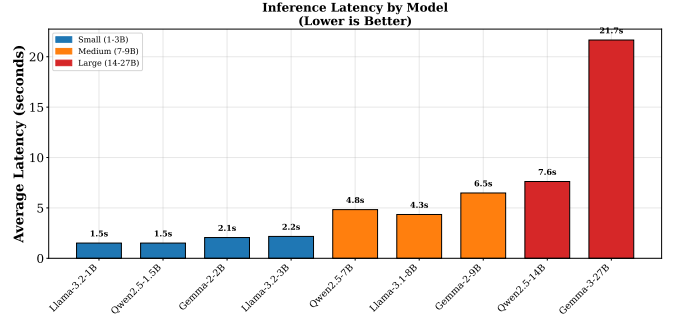
Ray Serve and KServe lack query-aware routing, resulting in high cold-start rates (35–38%). RouteLLM improves model selection but triggers cold starts 28% of the time without lifecycle awareness. FrugalGPT’s cascade adds latency from sequential model attempts. Pick and Spin achieves the lowest cold-start rate and best latency because Spin’s lifecycle state flows directly into Pick’s selection, routing around cold starts before they happen.

VIII. DISCUSSION

The medium-tier preference that Thompson Sampling converges to is a rational accuracy-per-GPU-hour optimization



(a) Throughput by model (higher is better).



(b) Inference latency by model (lower is better).

Fig. 2: Per-model performance from the static baseline across the nine-model pool on NVIDIA A100 GPUs with vLLM. Throughput drops 10 $\times$ and latency rises 14 $\times$ from small to large models, creating the routing opportunity that Pick exploits.

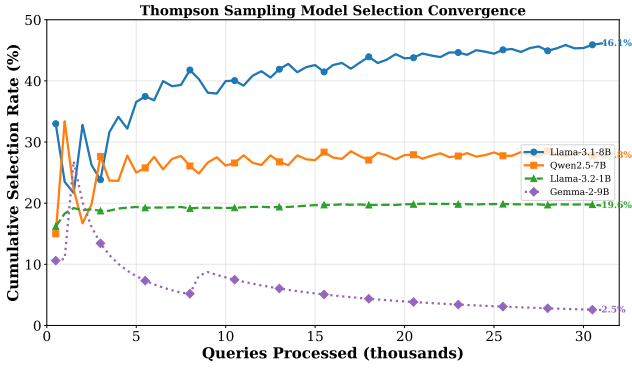


Fig. 3: Thompson Sampling convergence over 31,019 queries. Asymptotic selection rates (Llama-3.1-8B at 46.1%, Qwen2.5-7B at 27.8%) are higher than the actual routing distribution in Table I because the table includes early exploration queries and timeouts.

rather than an architectural bias: Qwen2.5-7B delivers 56.3% accuracy at 4.83 s of latency while Gemma-3-27B achieves only 56.6% at 21.66 s, a 4.5 $\times$ latency increase for a 0.3 point accuracy gain. The 49.7% overall routed accuracy in Table I reflects the routing trade-off and the 8.5% timeout rate, not a quality regression: each model’s per-routed-subset accuracy (e.g., Gemma-3-27B at 75.2% on the 633 queries Pick routed to it) shows that Pick concentrates the largest models on the queries that need them most.

Pick is designed as a pluggable component; Thompson Sampling can be replaced with RouteLLM, FrugalGPT, or a custom policy while retaining Spin’s lifecycle-aware scheduling. In production, where benchmark ground truth is unavailable, Pick is driven by implicit signals: HTTP errors, timeouts, malformed output, and optional user feedback. Thompson Sampling’s robustness to noisy and delayed rewards [31] makes it well suited to these signals, and validating routing

quality under them is a direct extension of this work. Shared-GPU serving with tensor parallelism, bursty arrival patterns with concurrency, and pools beyond 9–12 models (where hierarchical bandit structures become necessary) are natural extensions left to future work.

IX. CONCLUSION

Self-hosting multiple LLMs requires coordinating two decisions that existing tools handle separately: which model should serve each request, and how to manage model lifecycle state so that routing decisions do not trigger expensive cold starts. Pick and Spin addresses this integration gap through a co-designed pipeline built on Kubernetes. Pick routes queries based on complexity while incorporating lifecycle state from Spin; Spin manages model warm and cold states while anticipating routing patterns from Pick. This bidirectional feedback loop produces efficiency gains that neither component could achieve independently. At the core of Spin is the Cold Start State Machine, which provides orchestration logic that existing serving frameworks lack: rather than treating cold starts as unavoidable overhead, it actively tracks model readiness and exposes this to Pick at routing time, allowing the system to balance accuracy requirements against availability constraints in real time. Our evaluation on NRP Nautilus demonstrates 61% GPU utilization versus 31% for static deployment, a 46% reduction in GPU-hours, and a 9.2% per-query cold start rate. Thompson Sampling independently converges to favor medium-tier models, confirming that intelligent routing discovers the optimal accuracy-per-resource trade-off without manual configuration—validating that co-designing routing and lifecycle management yields measurable gains over treating the two problems in isolation.

ACKNOWLEDGMENT

This work is supported by NASA under grant NASA-AIST-21-0095 and the National Science Foundation under grant CNS-1846418.

REFERENCES

- [1] Private AI, “BYO LLM: Privacy concerns and other challenges with self hosting,” Blog post, Oct. 2023. [Online]. Available: <https://www.private-ai.com/en/blog/byo-llm>
- [2] P. Nguyen, “Cost-optimized ML on production: Autoscaling GPU nodes on Kubernetes to zero using KEDA,” Blog post, CodeLink, 2025. [Online]. Available: <https://www.codelink.io/blog/post/cost-optimized-ml-on-production-autoscaling-gpu-nodes-on-kubernetes-to-zero-using-keda>
- [3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes: Lessons from three container-management systems over a decade,” *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.
- [4] A. Verma, L. Pedrosa, M. R. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in *Proc. 10th European Conf. Computer Systems (EuroSys)*, Bordeaux, France, 2015, pp. 18:1–18:17.
- [5] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-LM: Training multi-billion parameter language models using model parallelism,” arXiv preprint arXiv:1909.08053, 2019.
- [6] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM SIGOPS Symp. Operating Systems Principles (SOSP)*, Koblenz, Germany, 2023, pp. 1–17.
- [7] Hugging Face Team, “Text generation inference (TGI): A toolkit for deploying and serving large language models,” Hugging Face, 2024. [Online]. Available: <https://github.com/huggingface/text-generation-inference>
- [8] R. Y. Aminabadi, S. Rajbhandari, M. Zhang, A. A. Awan, C. Li, D. Li, E. Zheng, J. Rasley, S. Smith, O. Ruwase, and Y. He, “DeepSpeed inference: Enabling efficient inference of transformer models at unprecedented scale,” arXiv preprint arXiv:2207.00032, 2022.
- [9] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, and X. Jin, “Fast distributed inference serving for large language models,” arXiv preprint arXiv:2305.05920, 2023.
- [10] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” arXiv preprint arXiv:2312.07104, 2024.
- [11] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *Proc. ICLR*, 2017.
- [12] NVIDIA, “Triton Inference Server: An open-source inference serving software,” 2024. [Online]. Available: <https://github.com/triton-inference-server/server>
- [13] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “RouteLLM: Learning to route LLMs with preference data,” arXiv preprint arXiv:2406.18665, 2024.
- [14] W. Jitkrittum, H. Narasimhan, A. S. Rawat, J. Juneja, C. Wang, Z. Wang, A. Go, C.-Y. Lee, P. Shenoy, R. Panigrahy, A. K. Menon, and S. Kumar, “Universal model routing for efficient LLM inference,” arXiv preprint arXiv:2502.08773, 2025.
- [15] Y.-K. Zhang, D.-C. Zhan, and H.-J. Ye, “Capability instruction tuning: A new paradigm for dynamic LLM routing,” arXiv preprint arXiv:2502.17282, 2025.
- [16] T. Xie, Y. Wu, Y. Luo, J. Ji, and X. Zheng, “Training-free multimodal large language model orchestration,” arXiv preprint arXiv:2508.10016, 2025.
- [17] P. Li and H. Yin, “Best practices for AI model inference configuration in Knative,” Blog post, Alibaba Cloud, Jul. 2024. [Online]. Available: https://www.alibabacloud.com/blog/aimodelinferenceknative_601454
- [18] K. Satzke, I. E. Akkus, R. Chen, I. Rimac, M. Stein, A. Beck, P. Aditya, M. Vanga, and V. Hilt, “Efficient GPU sharing for serverless workflows,” presented at TCSS562 course, University of Washington, 2022.
- [19] D. Crankshaw, G.-E. Sela, X. Mo, C. Zuber, I. Stoica, J. E. Gonzalez, and A. Tumanov, “InferLine: Latency-aware provisioning and scaling for prediction serving pipelines,” in *Proc. ACM Symp. Cloud Computing (SoCC)*, 2020.
- [20] H. Li, Z. Xu, G. Jiang, Y. Gao, Y. Sun, and H. Jin, “ModelSwitch: Fast switching of DNN models for cost-effective inference,” in *Proc. ACM Symp. Cloud Computing (SoCC)*, 2022.
- [21] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Pinto de Oliveira, J. Kaplan *et al.*, “Evaluating large language models trained on code,” arXiv preprint arXiv:2107.03374, 2021.
- [22] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, “Program synthesis with large language models,” arXiv preprint arXiv:2108.07732, 2021.
- [23] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” arXiv preprint arXiv:2110.14168, 2021.
- [24] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, “Measuring mathematical problem solving with the MATH dataset,” arXiv preprint arXiv:2103.03874, 2021.
- [25] S. Lin, J. Hilton, and O. Evans, “TruthfulQA: Measuring how models mimic human falsehoods,” arXiv preprint arXiv:2109.07958, 2022.
- [26] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? Try ARC, the AI2 reasoning challenge,” arXiv preprint arXiv:1803.05457, 2018.
- [27] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, “HellaSwag: Can a machine really finish your sentence?” in *Proc. 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [28] J. Mandel, D. Crawl, I. Altintas, S. Shankar, M. Zvyagin, and F. Wuerthwein, “Nautilus: A multi-tiered Kubernetes-based system for advanced analytics,” in *Proc. Practice and Experience in Advanced Research Computing (PEARC)*, 2020, pp. 165–172.
- [29] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “LLaMA: Open and efficient foundation language models,” arXiv preprint arXiv:2302.13971, 2023.
- [30] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” arXiv preprint arXiv:1910.01108, 2019.
- [31] D. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen, “A tutorial on Thompson sampling,” *Foundations and Trends in Machine Learning*, vol. 11, no. 1, pp. 1–96, 2018.
- [32] S. Bubeck and N. Cesa-Bianchi, “Regret analysis of stochastic and nonstochastic multi-armed bandit problems,” *Foundations and Trends in Machine Learning*, vol. 5, no. 1, pp. 1–122, 2012.
- [33] S. Moritz, R. Liaw, R. Nishihara, E. Gonzalez, J. E. Gonzalez, and I. Stoica, “Ray: A distributed framework for emerging AI applications,” in *Proc. 13th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2018, pp. 561–577.
- [34] A. Tsoukalas, “KServe: Serverless inferencing on Kubernetes,” *Cloud Native Computing Foundation*, 2021. [Online]. Available: <https://www.kubeflow.org/docs/external-add-ons/kserve/>
- [35] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, “Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider,” in *Proc. 2020 USENIX Annual Technical Conf.*, 2020, pp. 205–218.
- [36] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng *et al.*, “Qwen technical report,” arXiv preprint arXiv:2309.16609, 2023.
- [37] Google DeepMind, “Gemma: Open models based on Gemini research and technology,” Technical Report, 2024. [Online]. Available: <https://www.kaggle.com/models/google/gemma>
- [38] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen, “MMLU-Pro: A more robust and challenging multi-task language understanding benchmark,” arXiv preprint arXiv:2406.01574, 2024.
- [39] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” arXiv preprint arXiv:2306.05685, 2023.
- [40] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” arXiv preprint arXiv:2305.05176, 2023.